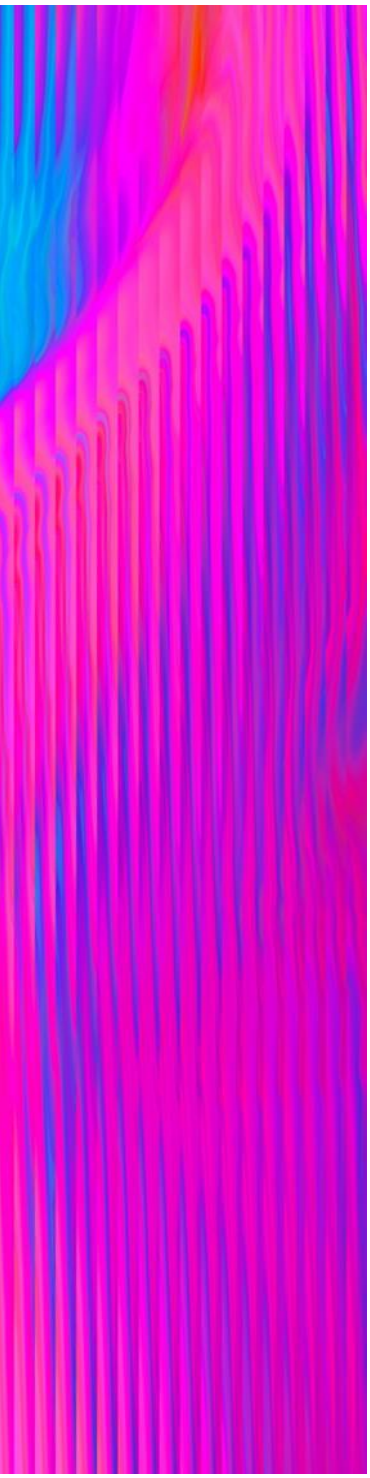
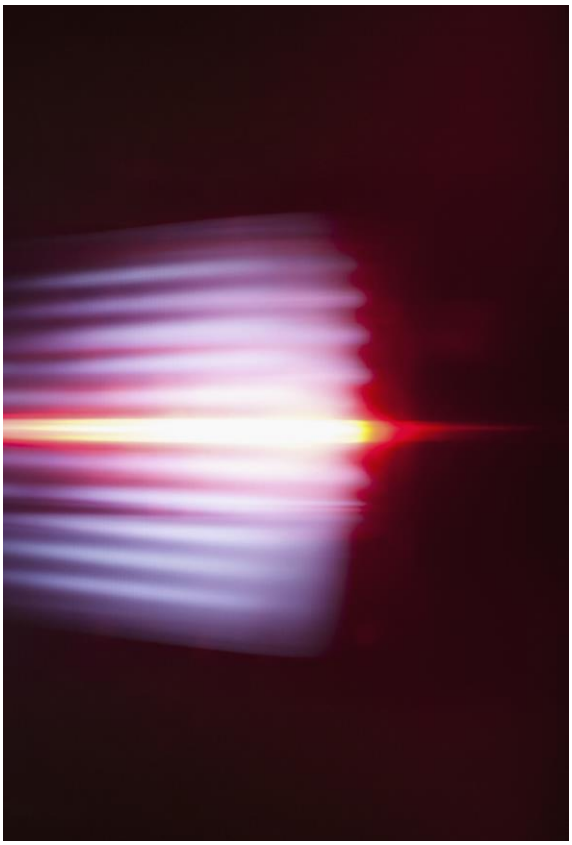


TIVIDY: WORLFLOW MANAGEMENT SYSTEM

BY O(￣▽￣)D WE GOT THIS

WE HOPE YOU LIKE OUR SYSTEM!!



PREPARED BY

- U25312040: MOHAMMAD H JAFARI
- U25033931: ZANDER SPIES
- U24578071: NOKHAYA P FOLOTI
- U05175552: LEBOGANG NXUMALO
- U23586975:EMMANUEL BOATENG

GitHub link:
https://github.com/MH-Multy/COS214_Prac6.git

TABLE OF CONTENTS:

Research & Findings 3

Scope of TIVIDY 4

Chosen scenarios 5

Updated requirements and UML Diagrams 7

Implementation Details & design decision history 9

Build/run/test instructions 11

Git and engineering evidence .....

Team contribution statements .....

Workflow-management research findings and references 🔬

What is a workflow and workflow management system?

“The workflow management process focuses on organizing and optimizing the tasks needed to complete business activities.” This usually involves team coordination and progress tracking. By utilizing a workflow management, businesses can increase efficiency and eliminate bottlenecks. “A workflow refers to the set of tasks needed to complete a specific business process. Workflow processes should clearly outline how work moves from one stage to another, often specifying who performs each task and in what order”. A workflow management software can provide tools to address these challenges and keep businesses productive.

Workflow task routing

In a workflow, tasks must always be able to reach their intended target and/or handled in a suitable manner when they get accepted/rejected/waiting. It is the workflows job to insure this is the case and allow for tracking of certain tasks.

Workflow definition vs instance

A workflow definition specifies, which actions should be taken, when and under what circumstances. The execution of these actions is called a workflow instance, “a single workflow definition can have multiple workflow instances”.

What makes a workflow engine reusable?

The code itself and functionality mostly remain the same and the core concept does not change. The definitions and process however can vary depending on business/task that is intended to be completed. This means most functionalities can become their own parts and only the necessary parts can be reassembled to run the desired task, meaning that the base functionality can be hard coded and the branches and sub process be configurable.

Changes to workflow while work is processing

Due to the nature of most processes, it is best to let the current fork flow finish and halt all new instances and the update it. Updating while processing may lead to unexpected errors or results. On large workflows, it is recommended to roll out changes slowly by sections so that the potential damage is reduced or removed. This is especially important when working with old systems and updating them drastically.

References:

https://help.sap.com/docs/workflow-capability/workflow-cloud-foundry/workflow-definition-versus-workflow-instance

https://www.sap.com/resources/workflow-management

Scope of TIVIDY 🔭

TIVIDY is a generic workflow management system designed to define, execute and manage workflows. The system will allow organisations to create, execute, manage and assign work to appropriate participants. It will also allow organisations to record and change the workflow requirements at any time depending on their need. The overall design will not be bound to any one organisation but rather customized to fit their need.



System parts that may vary 🔎

The following will vary depending on organisation to organisation:

- ***Assignment rules***: work may be assigned to a particular person, role or participant based on workload or other workflow information
- ***Routing rules***: The next activity is dependent on organization needs and requirements
- ***Approval/Escalation rules***: Approval numbers or time to reply is dependent on organisation along with who it must escalate too.
- ***Workflow activities***: Combinations of process may vary depending on organisation
- ***Events and triggers***: Different workflows responses may vary depending on the internal/external events
- ***External integration***: Workflow may interact with different external systems
- ***Failure handlings***: Different workflows require different recovery and exception behaviour.

OUR CHOSEN SCENARIOS

Laboratory testing workflow system

Participants:

- Researcher (Requester)
- Lab Manager/ Head Researcher (Approver)
- Intern (restricted actor)

1. **Submitting a Test Request**

A researcher fills out a form to book lab space and equipment (like a specialized biosafety cabinet). They specify what experiment they want to run and list the details, including danger levels or safety ratings.

2. **Automated Safety Checks**

Before anyone reviews the request by hand, the system automatically runs essential checks:

- Does the form have all mandatory information filled in?
- Is the person an active, recognized worker in the organization's staff directory?
- Does this person have the right training and safety clearance to use the requested machine?
- Does the test stay within safe limits (such as not exceeding hazardous chemical or biohazard limits)?

3. **Supervisory Review**

- If any safety check fails, the system blocks the request immediately and explains why.
- If everything checks out, the request moves to the head lab manager's review list.
- The manager can approve the test to let work begin, or decline it.

4. **Running the Test & Safety Halts**

- Once approved, the researcher enters the lab and begins their work steps.
- If a problem arises during the test – such as an equipment fault or a containment alert – the manager can immediately suspend the test, pausing all work on that experiment until it is safe to resume.

5. **Logging Results & Tracking Changes**

- Throughout the process, the researcher writes down their findings and updates daily or weekly progress logs.
- Every edit is saved with its revision history, allowing researchers to review earlier drafts or undo mistakes without losing notes.
- The system keeps a complete, tamper-proof history of every action taken – who submitted the test, who approved it, and any pauses that occurred – for safety records.

Loan application workflow system

Participants:
- Borrowing Applicant (Requester)
- Credit Officer
- Branch Manager
- High risk applicant (Ineligible actor)

1. **Applying for Credit**

A client fills out an application to borrow money. They state the loan amount they need, their income details, and provide assets – such as a house deed or vehicle title – as collateral to back the loan.

2. **Automated Financial Pre-Screening**

Before any human spends time on the file, the system runs immediate automated checks:

- Does the application include all required fields, including collateral details?
- Is the applicant a real person verified in the central identity registry?
- Does the assigned credit officer have the authorization to review loans of this size?
- Does the applicant stay within safe borrowing boundaries, such as keeping their debt-to-income ratio below safe limits?

3. **Officer Review and Decisions**

- If any automated safety check fails, the application is stopped right away, and the borrower is informed why.
- If everything checks out, the application lands on the desk of an assigned credit officer or committee.
- The officer evaluates the borrower's credit score and collateral value to officially approve or decline the loan. Small, low-risk requests can also be set to auto-approve if no red flags are found within a set review period.

4. **Safety Pauses and Discrepancies**

- If a problem arises while the loan is being processed – such as a title dispute on the collateral or a fraud alert – the officer can immediately suspend the application.
- Suspending the loan pauses all payouts and stops further actions until the problem is investigated and cleared.

5. **Risk Analysis Reports and Audit Records**

- The credit officer writes an ongoing risk analysis and assessment report detailing whether the applicant can afford the payments.
- The system saves every draft of this report, allowing officers to look back at earlier versions or revert mistakes.

- A permanent, unchangeable record tracks every single step – from submission to automated checks, to approvals or pauses – ensuring full compliance with financial rules.

Updated requirements, UML Diagrams and Patterns

- ## Updated requirements

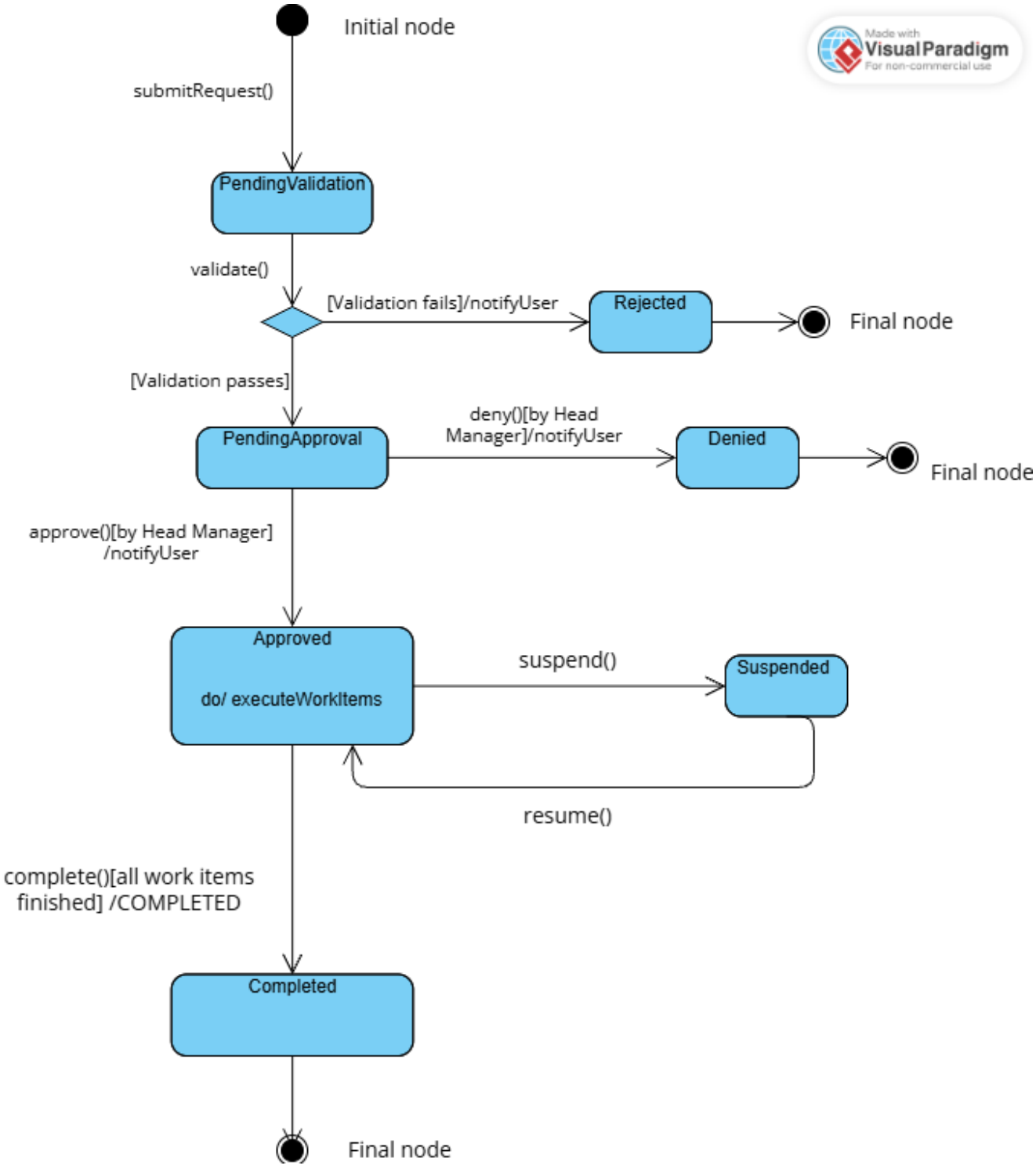
1. Updating chain for Chain of responsibility
-Initially the chain was supposed to pass responsibility human approval decisions, timeouts and manager signoff down the pipeline.
-Now human approval was moved to the Strategy patterns, and CoR enforces data validation and clearance invariants

2.Reducing States
-Initially included user-dependent states, such as usingEquipement and Returned, which bloated the state pattern and included unnecessary states. Instead, it was formalized to 7 industry-neutral states: *PendingValidation, PendingApproval, Approved, InProgress, Suspended, Completed, and Rejected.*

3. Repurposing the Strategy pattern
- Strategy was designed exclusively for work item and task routing via an AssignmentStrategy interface (RoleBasedAssignment, WorkloadBalancingAssignment under a TaskRouter).
-For the final system, dynamic task management via TaskRouter was simplified, the Strategy pattern now is used to select the ApprovalPolicy, dictating runtime governance rules

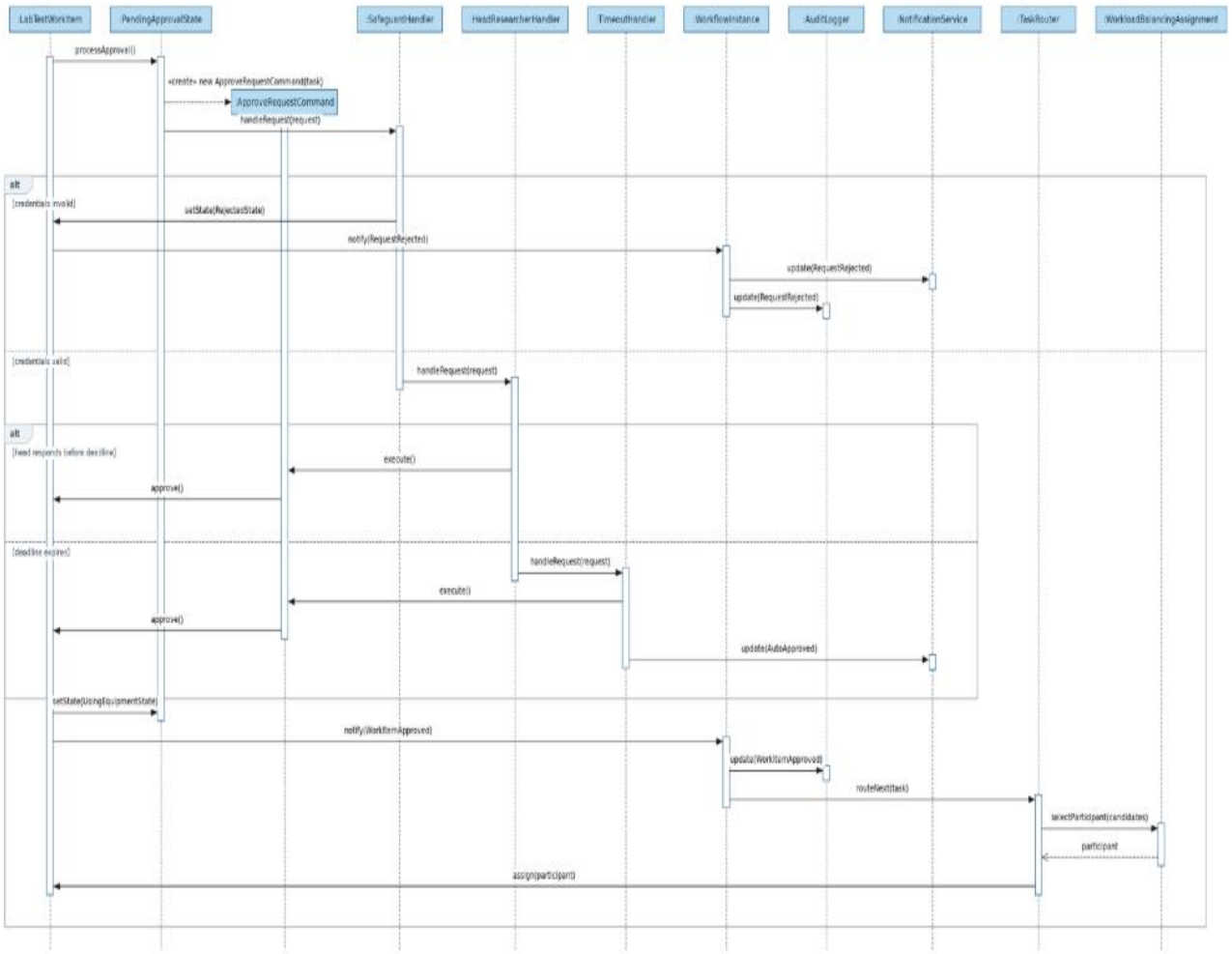
- UML Diagrams

STATE DIAGRAM

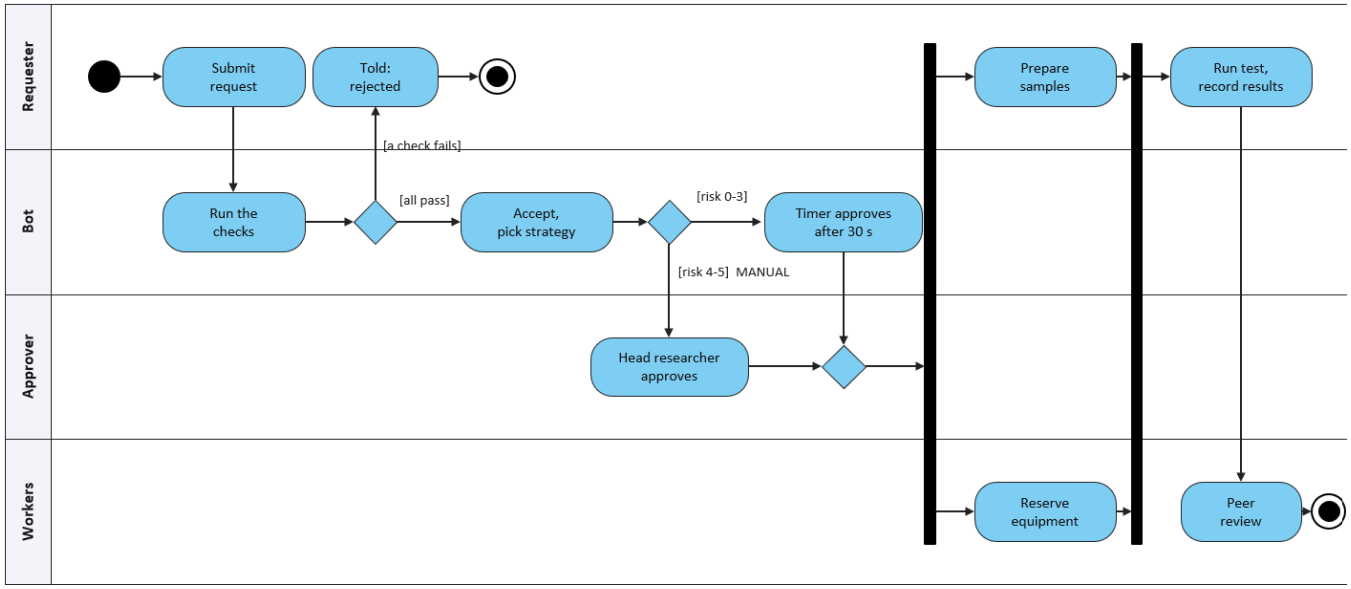


Sequence diagrams

Sequence Diagram 2 — Approval, Escalation and Routing



Activity Diagrams



Implementation Details & design decision history

In the initial Practical 6 planning model, business invariant validation, human review hierarchies, and timeout escalations were commingled within a single ApprovalHandler hierarchy under the Chain of Responsibility pattern. This coupled automated schema checks with long-running human interactions. Furthermore, the Memento pattern was originally conceptualized as an all-encompassing system checkpoint (WorkflowMemento / CheckpointManager) aimed at serializing entire runtime instances.

In practice, full-engine snapshots created severe pointer aliasing hazards and memory leaks across dynamic state pointers.

Key Architectural Improvements

- **Separating Automated Checks from Human Decisions**

We separated instant software checks from human manager reviews. The validation chain runs first, automatically verifying forms and safety credentials before anything else moves forward. Slower processes – such as waiting for a manager's signature, group voting rules, or auto-approval timers – were moved into their own dedicated action and policy modules.

- **Saving Draft Snapshots for Reports Only**

Instead of freezing and saving the entire system engine during an operation, we narrowed our snapshot system down to the actual work documents. Whenever someone edits a laboratory log or adds loan assessment notes to a report, the system saves an unchangeable snapshot of that revision. This allows users to review past revisions or revert changes easily without risking the engine's memory.

- **One Universal Connector for Older Databases**

Rather than writing custom code for every external database, we created one universal directory format. Our connector takes raw text strings from legacy computer databases (such as employee status and role lists) and translates them into clean true-or-false answers. This allows the exact same verification logic to work across hospital staff lists and bank customer records without altering the core software.

- **Enforcing 7 Clear Lifecycle Stages**

We replaced loose text labels with seven distinct, strictly enforced stages: Waiting for Validation, Waiting for Approval, Approved, In Progress, Suspended, Completed, and Rejected. The system explicitly blocks invalid skips – such as trying to complete a task that has been suspended or trying to approve an application before safety checks have passed.

- **Automatic Event Broadcasting and Alerts**

Every time a request changes stages, it automatically sends an update to everyone who needs to know. A permanent history log records everything for compliance, the person who made the request receives private updates in their inbox, and supervisors get notified whenever a task is waiting for their review.

Subsystem Implementation & Non-Obvious Logic

1. **Chain of Responsibility: Traversal Modes & Selective Participation**

The ValidationHandler base defines handle(const WorkflowInstance&, ChainExecutionMode):

- Selective Participation (canHandle()): Handlers dynamically inspect payloads first. For example, ThresholdHandler skips execution and passes control downstream if its target numerical attribute is absent, avoiding false errors.
- Dual Traversal Modes: FAIL_FAST aborts immediately on the first critical violation to save resources; AUDIT_ACCUMULATE runs the full chain, aggregating all warnings and errors into a ValidationResult for compliance review.

2. **Command Invariant Boundaries & Reversible History**

The CommandInvoker manages history stacks while enforcing strict audit boundaries:

- Reversible Actions: Operational tasks (CompleteWorkItemCommand) and deliverable amendments (EditReportCommand) support full rollback via undo().
- Non-Reversible Boundaries: Formal regulatory decisions (ApproveCommand, RejectCommand) cannot be undone, preventing unauthorized status tampering.

3. **WorkItem Composite & Filtered Iteration**

Task execution is structured as a WorkItem hierarchy:

- DepthFirstIterator: Recursively walks the entire composite tree for administrative enumeration and auditing.
- AvailableWorkIterator: Applies runtime filtering to yield only unblocked leaf tasks whose prerequisites are satisfied, preventing out-of-order execution in concurrent branches.

4. **Observer Event Routing & Targeted Inboxes**

To prevent alert fatigue, observers filter incoming WorkflowEvent broadcasts:

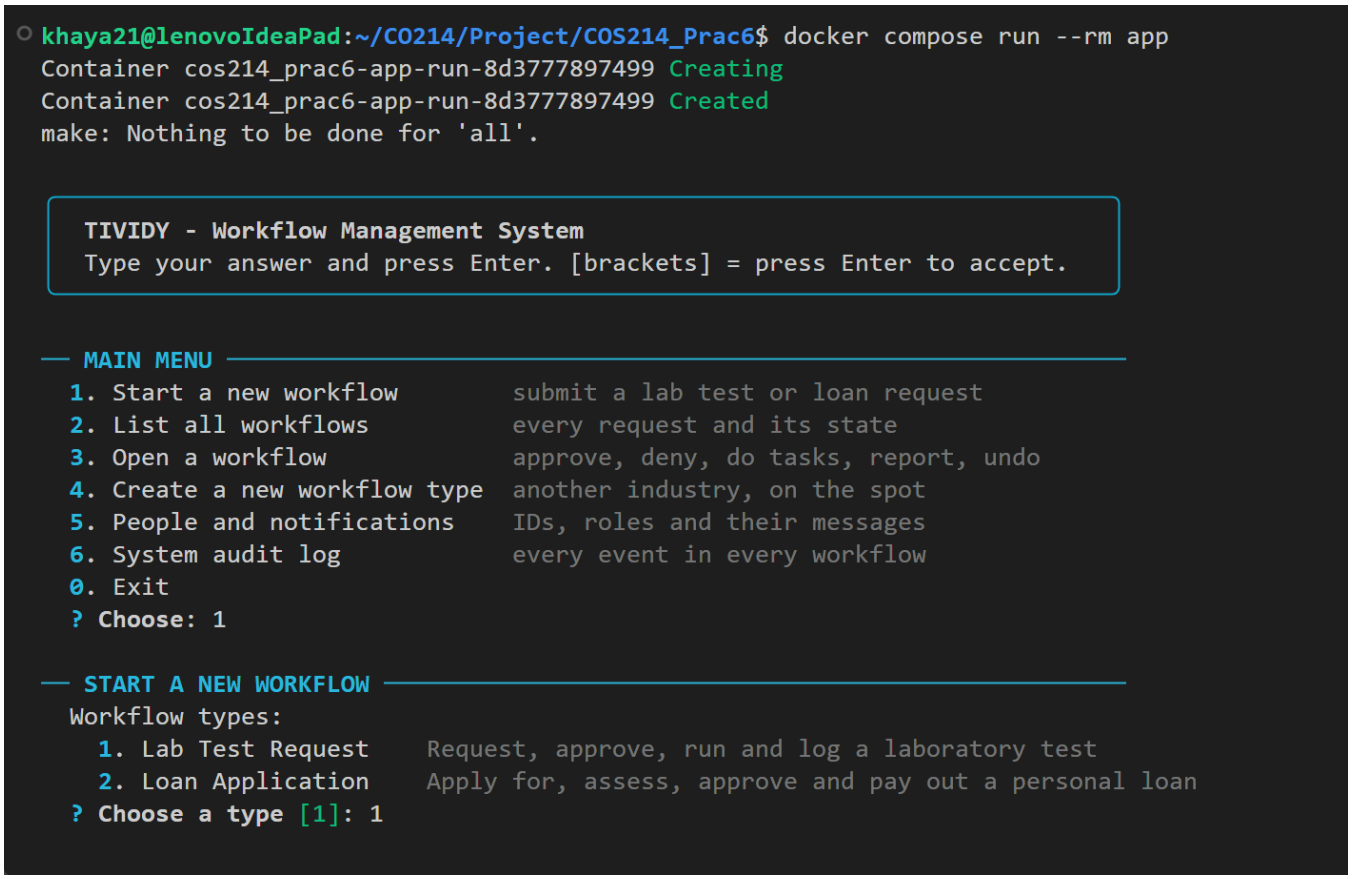
- AuditLogger: Records every event chronologically into an immutable system audit trail.
- RequesterObserver: Routes relevant status updates to the specific applicant's private inbox (targetRequesterId_).
- ApproverObserver: Discards routine progress updates and queues only actionable supervisory checkpoints (such as pending approvals or emergency halts).

Build/run/test instructions

- **Compilation:** make
- **Automated Testing:** make test (Executed via **doctest.h** test suites)
- **Documentation:** doxygen Doxyfile
- **Containerization:** docker compose up --build

Evidence

- **Docker compose**



- **Valgrind**

```
khaya21@lenovoIdeaPad:~/CO214/Project/COS214_Prac6$ docker compose run --rm app valgrind --lea
k-check=full ./main
  5. People and notifications    IDs, roles and their messages
  6. System audit log            every event in every workflow
  0. Exit
  ? Choose: 0

  Goodbye.
==1==
==1== HEAP SUMMARY:
==1==     in use at exit: 122,880 bytes in 6 blocks
==1==   total heap usage: 251 allocs, 245 frees, 212,067 bytes allocated
==1==
==1== LEAK SUMMARY:
==1==    definitely lost: 0 bytes in 0 blocks
==1==    indirectly lost: 0 bytes in 0 blocks
==1==      possibly lost: 0 bytes in 0 blocks
==1==    still reachable: 122,880 bytes in 6 blocks
==1==         suppressed: 0 bytes in 0 blocks
==1== Reachable blocks (those to which a pointer was found) are not shown.
==1== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
khaya21@lenovoIdeaPad:~/CO214/Project/COS214_Prac6$
```

- **Doctest summary**

```
khaya21@lenovoIdeaPad:~/CO214/Project/COS214_Prac6$ docker compose run --rm app make test
./run_tests
[doctest] doctest version is "2.5.0"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 109 | 109 passed | 0 failed | 0 skipped
[doctest] assertions: 383 | 383 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Team Contributions

– Nokhaya Princess Foloti:

Implemented the **Chain of Responsibility**, **Adapter**, **Observer**, and **Iterator** design patterns for the project. In addition, I configured the **Docker** environment, set up the **Doxygen** code documentation, and authored the final PDF report.

– Mohammad Hossein Jafari:

Implemented the **State** and **Strategy** design patterns for the project. I also assisted

with the final design of the **Power Point** and managed the **Git Hub** approval and permissions on branches.

- Emmanuel Boateng:

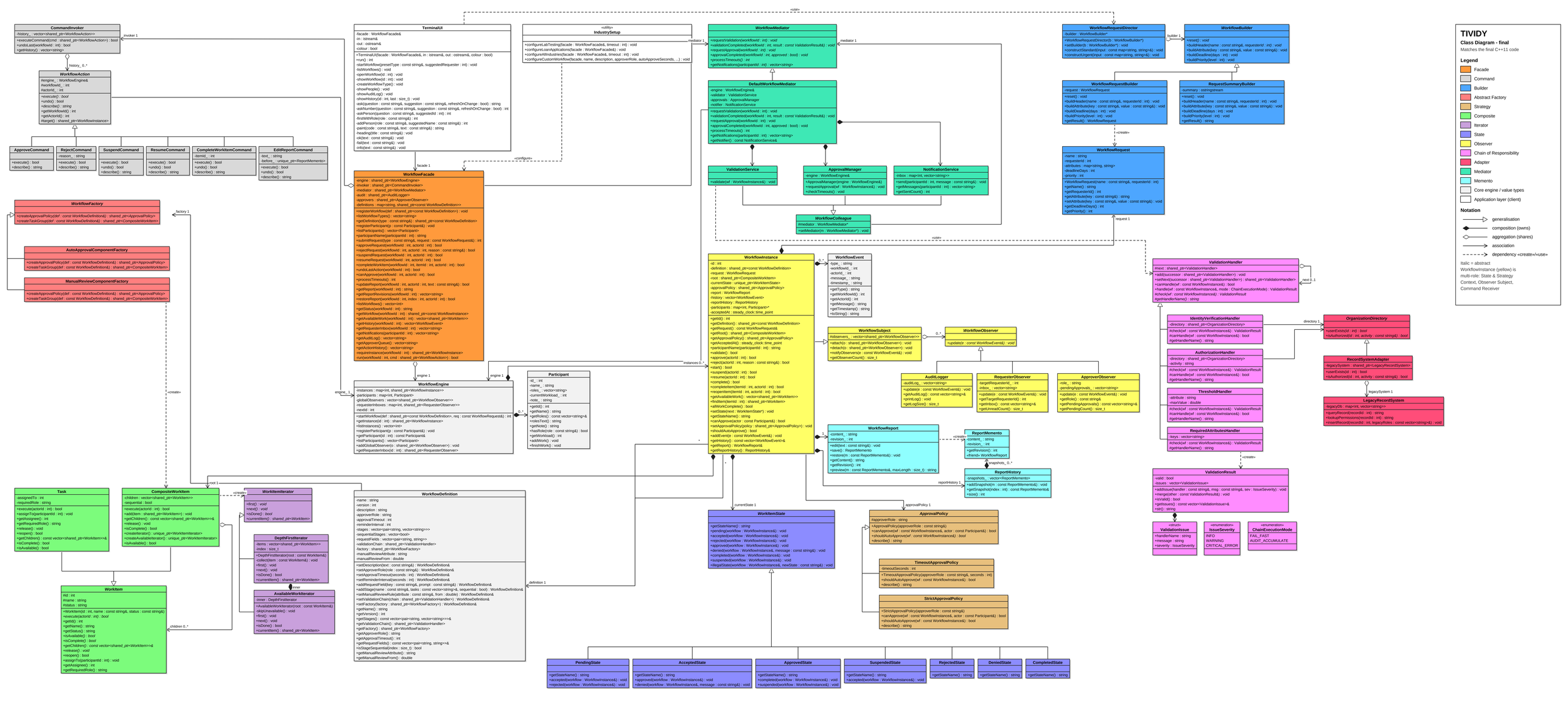
  Created the final **Class Diagram**. Implemented **Facade**, **Command** and **Memento** Design pattern as well as the **Final TUI**. I also created the branches for everyone to work on.

- Zander Spies:

  Implemented the **Composite** and **Builder** design patterns. I also assisted with creating the initial **Class Diagram**.

- Lebogang Nxumalo:

  Implemented the **Abstract Factory** and **Mediator** design patterns. I also implemented the initial design of the **Power Point**.


TIVIDY
Class Diagram - final
Matches the final C++11 code
Legend
Facade
Command
Builder
Abstract Factory
Strategy
Composite
Iterator
State
Observer
Chain of Responsibility
Adapter
Mediator
Memento
Core engine / value types
Application layer (client)
Notation
generalisation
composition (owns)
aggregation (shares)
association
dependency «create»/«use»
Italic = abstract
WorkflowInstance (yellow) is multi-role: State & Strategy Context, Observer Subject, Command Receiver
CommandInvoker
-history_ : vector<shared_ptr<WorkflowAction>>
+executeCommand(cmd : shared_ptr<WorkflowAction>) : bool
+undoLast(workflowId : int) : bool
+getHistory() : vector<string>
«invoke»
TerminalUI
-facade : WorkflowFacade&
-in : istream&
-out : ostream&
-colour : bool
+TerminalUI(facade : WorkflowFacade&, in : istream&, out : ostream&, colour : bool)
+run() : int
-startWorkflow(presetType : const string&, suggestedRequester : int) : void
-listWorkflows() : void
-openWorkflow(id : int) : void
-showWorkflow(id : int) : void
-createWorkflowType() : void
-showPeople() : void
-showAuditLog() : void
-showHistory(id : int, last : size_t) : void
-ask(question : const string&, suggestion : const string&, refreshOnChange : bool) : string
-askNumber(question : const string&, suggestion : const string&, refreshOnChange : bool) : int
-askPerson(question : const string&, suggestedId : int) : int
-fireWriteRole(role : const string&) : int
-addPerson(role : const string&, suggestedName : const string&) : int
-paint(code : const string&, text : const string&) : string
-heading(title : const string&) : void
-ok(text : const string&) : void
-fail(text : const string&) : void
-info(text : const string&) : void
«utility»
IndustrySetup
+configureLabTesting(facade : WorkflowFacade&, timeout : int) : void
+configureLoanApplications(facade : WorkflowFacade&) : void
+configureAllIndustries(facade : WorkflowFacade&, timeout : int) : void
+configureCustomWorkflow(facade, name, description, approverRole, autoApproveSeconds, ...) : void
«configure»
WorkflowMediator
+requestValidation(workflowId : int) : void
+validationCompleted(workflowId : int, result : const ValidationResult&) : void
+requestApproval(workflowId : int) : void
+approvalCompleted(workflowId : int, approved : bool) : void
+processTimeouts() : int
+getNotifications(participantId : int) : vector<string>
mediator 1
DefaultWorkflowMediator
-engine : WorkflowEngine&
-validator : ValidationService
-approvals : ApprovalManager
-notifier : NotificationService
+requestValidation(workflowId : int) : void
+validationCompleted(workflowId : int, result : const ValidationResult&) : void
+requestApproval(workflowId : int) : void
+approvalCompleted(workflowId : int, approved : bool) : void
+processTimeouts() : int
+getNotifications(participantId : int) : vector<string>
+getNotifier() : const NotificationService&
ValidationService
+validate(wf : WorkflowInstance&) : void
ApprovalManager
-engine : WorkflowEngine&
+ApprovalManager(engine : WorkflowEngine&)
+requestApproval(wf : WorkflowInstance&) : void
+checkTimeouts() : void
NotificationService
-inbox : map<int, vector<string>>
+send(participantId : int, message : const string&) : void
+getMessages(participantId : int) : vector<string>
+getSentCount() : int
WorkflowColleague
#mediator : WorkflowMediator*
+setMediator(m : WorkflowMediator*) : void
WorkflowRequestDirector
-builder : WorkflowBuilder*
+WorkflowRequestDirector(b : WorkflowBuilder*)
+setBuilder(b : WorkflowBuilder*) : void
+constructStandard(input : const map<string, string>&) : void
+constructUrgent(input : const map<string, string>&) : void
builder 1
WorkflowBuilder
+reset() : void
+buildHeader(name : const string&, requesterId : int) : void
+buildAttribute(key : const string&, value : const string&) : void
+buildDeadline(days : int) : void
+buildPriority(level : int) : void
WorkflowRequestBuilder
-request : WorkflowRequest
+reset() : void
+buildHeader(name : const string&, requesterId : int) : void
+buildAttribute(key : const string&, value : const string&) : void
+buildDeadline(days : int) : void
+buildPriority(level : int) : void
+getResult() : WorkflowRequest
RequestSummaryBuilder
-summary : ostringstream
+reset() : void
+buildHeader(name : const string&, requesterId : int) : void
+buildAttribute(key : const string&, value : const string&) : void
+buildDeadline(days : int) : void
+buildPriority(level : int) : void
+getResult() : string
«create»
WorkflowRequest
-name : string
-requesterId : int
-attributes : map<string, string>
-deadlineDays : int
-priority : int
+WorkflowRequest(name : const string&, requesterId : int)
+getName() : string
+getRequesterId() : int
+getAttribute(key : const string&) : string
+setAttribute(key : const string&, value : const string&) : void
+getDeadlineDays() : int
+getPriority() : int
request 1
WorkflowFacade
-engine : shared_ptr<WorkflowEngine>
-invoker : shared_ptr<CommandInvoker>
-mediator : shared_ptr<WorkflowMediator>
-audit : shared_ptr<AuditLogger>
-approvers : shared_ptr<ApproverObserver>
-definitions : map<string, shared_ptr<const WorkflowDefinition>>
+registerWorkflow(def : shared_ptr<const WorkflowDefinition>) : void
+listWorkflowTypes() : vector<string>
+getDefinition(type : const string&) : shared_ptr<const WorkflowDefinition>
+registerParticipant(p : const Participant&) : void
+listParticipants() : vector<Participant>
+participantName(participantId : int) : string
+submitRequest(type : const string&, request : const WorkflowRequest&) : int
+approveRequest(workflowId : int, actorId : int) : bool
+rejectRequest(workflowId : int, actorId : int, reason : const string&) : bool
+suspendRequest(workflowId : int, actorId : int) : bool
+resumeRequest(workflowId : int, actorId : int) : bool
+completeWorkItem(workflowId : int, itemId : int, actorId : int) : bool
+undoLastAction(workflowId : int) : bool
+canApprove(workflowId : int, actorId : int) : bool
+processTimeouts() : int
+updateReport(workflowId : int, actorId : int, text : const string&) : bool
+getReport(workflowId : int) : string
+getReportRevisions(workflowId : int) : vector<string>
+restoreReport(workflowId : int, index : int, actorId : int) : bool
+listWorkflows() : vector<int>
+getStatus(workflowId : int) : string
+getWorkflowOwner(workflowId : int) : shared_ptr<const WorkflowInstance>
+getAvailableWork(workflowId : int) : vector<shared_ptr<WorkItem>>
+getHistory(workflowId : int) : vector<WorkflowEvent>
+getRequesterInbox(workflowId : int) : vector<string>
+getNotifications(participantId : int) : vector<string>
+getAuditLog() : vector<string>
+getApproverQueue() : vector<string>
+getActionHistory() : vector<string>
+requireInstance(workflowId : int) : shared_ptr<WorkflowInstance>
+run(workflowId : int, cmd : shared_ptr<WorkflowAction>) : bool
facade 1
«configure»
WorkflowFactory
+createApprovalPolicy(def : const WorkflowDefinition&) : shared_ptr<ApprovalPolicy>
+createTaskGroup(def : const WorkflowDefinition&) : shared_ptr<CompositeWorkItem>
factory 1
AutoApprovalComponentFactory
+createApprovalPolicy(def : const WorkflowDefinition&) : shared_ptr<ApprovalPolicy>
+createTaskGroup(def : const WorkflowDefinition&) : shared_ptr<CompositeWorkItem>
ManualReviewComponentFactory
+createApprovalPolicy(def : const WorkflowDefinition&) : shared_ptr<ApprovalPolicy>
+createTaskGroup(def : const WorkflowDefinition&) : shared_ptr<CompositeWorkItem>
«create»
WorkflowInstance
-id : int
-definition : shared_ptr<const WorkflowDefinition>
-request : WorkflowRequest
-root : shared_ptr<CompositeWorkItem>
-currentState : unique_ptr<WorkItemState>
-approvalPolicy : shared_ptr<ApprovalPolicy>
-report : WorkflowReport
-history : vector<WorkflowEvent>
-reportHistory : ReportHistory
-participants : map<int, Participant>*
-acceptedAt : steady_clock::time_point
+getId() : int
+getDefinition() : shared_ptr<const WorkflowDefinition>
+getRequest() : const WorkflowRequest&
+getRoot() : shared_ptr<CompositeWorkItem>
+getApprovalPolicy() : shared_ptr<ApprovalPolicy>
+getAcceptedAt() : steady_clock::time_point
+participantName(participantId : int) : string
+validate() : bool
+approve(actorId : int) : bool
+reject(actorId : int, reason : const string&) : bool
+start() : bool
+suspend(actorId : int) : bool
+resume(actorId : int) : bool
+complete() : bool
+completeItem(itemId : int, actorId : int) : bool
+reopenItem(itemId : int, actorId : int) : bool
+getAvailableWork() : vector<shared_ptr<WorkItem>>
+findItem(itemId : int) : shared_ptr<WorkItem>
+allWorkComplete() : bool
+setState(next : WorkItemState*) : void
+getStateName() : string
+canApprove(actor : const Participant&) : bool
+setApprovalPolicy(policy : shared_ptr<ApprovalPolicy>) : void
+shouldAutoApprove() : bool
+addEvent(e : const WorkflowEvent&) : void
+getHistory() : const vector<WorkflowEvent>&
+getReport() : WorkflowReport&
+getReportHistory() : ReportHistory&
WorkflowEvent
-type_ : string
-workflowId_ : int
-actorId_ : int
-message_ : string
-timestamp_ : string
+getType() : string
+getWorkflowId() : int
+getActorId() : int
+getMessage() : string
+getTimestamp() : string
+toString() : string
0..*
WorkflowSubject
#observers_ : vector<shared_ptr<WorkflowObserver>>
+attach(o : shared_ptr<WorkflowObserver>) : void
+detach(o : shared_ptr<WorkflowObserver>) : void
+notifyObservers(e : const WorkflowEvent&) : void
+getObserverCount() : size_t
WorkflowObserver
+update(e : const WorkflowEvent&) : void
0..*
AuditLogger
-auditLog_ : vector<string>
+update(e : const WorkflowEvent&) : void
+getAuditLog() : const vector<string>&
+printLog() : void
+getLogSize() : size_t
RequesterObserver
-targetRequesterId_ : int
-inbox_ : vector<string>
+update(e : const WorkflowEvent&) : void
+getTargetRequesterId() : int
+getInbox() : const vector<string>&
+getUnreadCount() : size_t
ApproverObserver
-role_ : string
-pendingApprovals_ : vector<string>
+update(e : const WorkflowEvent&) : void
+getRole() : const string&
+getPendingApprovals() : const vector<string>&
+getPendingCount() : size_t
WorkflowReport
-content_ : string
-revision_ : int
+edit(text : const string&) : void
+save() : ReportMemento
+restore(m : const ReportMemento&) : void
+getContent() : string
+getRevision() : int
+preview(m : const ReportMemento&, maxLength : size_t) : string
«create»
ReportMemento
-content_ : string
-revision_ : int
+getRevision() : int
+friend> WorkflowReport
1
snapshots_ 0..*
ReportHistory
-snapshots_ : vector<ReportMemento>
+addSnapshot(m : const ReportMemento&) : void
+getSnapshot(index : int) : const ReportMemento&
+size() : int
reportHistory 1
ValidationHandler
#next : shared_ptr<ValidationHandler>
+add(successor : shared_ptr<ValidationHandler>) : void
+setNext(successor : shared_ptr<ValidationHandler>) : shared_ptr<ValidationHandler>
+canHandle(wf : const WorkflowInstance&) : bool
+handle(wf : const WorkflowInstance&, mode : ChainExecutionMode) : ValidationResult
#check(wf : const WorkflowInstance&) : ValidationResult
#getHandlerName() : string
next 0..1
IdentityVerificationHandler
-directory : shared_ptr<OrganizationDirectory>
#check(wf : const WorkflowInstance&) : ValidationResult
#canHandle(wf : const WorkflowInstance&) : bool
#getHandlerName() : string
directory 1
OrganizationDirectory
+userExists(id : int) : bool
+isAuthorized(id : int, activity : const string&) : bool
AuthorizationHandler
-directory : shared_ptr<OrganizationDirectory>
-activity : string
#check(wf : const WorkflowInstance&) : ValidationResult
#canHandle(wf : const WorkflowInstance&) : bool
#getHandlerName() : string
RecordSystemAdapter
-legacySystem : shared_ptr<LegacyRecordSystem>
+userExists(id : int) : bool
+isAuthorized(id : int, activity : const string&) : bool
legacySystem 1
ThresholdHandler
-attribute : string
-maxValue : double
#check(wf : const WorkflowInstance&) : ValidationResult
#canHandle(wf : const WorkflowInstance&) : bool
#getHandlerName() : string
LegacyRecordSystem
-legacyDb : map<int, vector<string>>
+queryRecord(recordId : int) : string
+lookupPermissions(recordId : int) : string
+insertRecord(recordId : int, legacyRoles : const vector<string>&) : void
RequiredAttributesHandler
-keys : vector<string>
#check(wf : const WorkflowInstance&) : ValidationResult
#getHandlerName() : string
«create»
ValidationResult
-valid : bool
-issues : vector<ValidationIssue>
+addIssue(handler : const string&, msg : const string&, sev : IssueSeverity) : void
+merge(other : const ValidationResult&) : void
+isValid() : bool
+getIssues() : const vector<ValidationIssue>&
+str() : string
«struct»
ValidationIssue
+handlerName : string
+message : string
+severity : IssueSeverity
«enumeration»
IssueSeverity
INFO
WARNING
CRITICAL_ERROR
«enumeration»
ChainExecutionMode
FAIL_FAST
AUDIT_ACCUMULATE
Participant
-id_ : int
-name_ : string
-roles_ : vector<string>
-currentWorkload_ : int
-role_ : string
+getId() : int
+getName() : string
+getRoles() : const vector<string>&
+rolesText() : string
+getNote() : string
+hasRole(role : const string&) : bool
+getWorkload() : int
+addWork() : void
+finishWork() : void
engine 1
instances 0..*
WorkflowEngine
-instances : map<int, shared_ptr<WorkflowInstance>>
-participants : map<int, Participant>
-globalObservers : vector<shared_ptr<WorkflowObserver>>
-requesterInboxes : map<int, shared_ptr<RequesterObserver>>
-nextId : int
+startWorkflow(def : shared_ptr<const WorkflowDefinition>, req : const WorkflowRequest&) : int
+getInstance(id : int) : shared_ptr<WorkflowInstance>
+listInstances() : vector<int>
+registerParticipant(p : const Participant&) : void
+getParticipant(id : int) : const Participant&
+listParticipants() : vector<Participant>
+addGlobalObserver(o : shared_ptr<WorkflowObserver>) : void
+getRequesterInbox(id : int) : shared_ptr<RequesterObserver>
engine_ 1
0..*
«create»
Task
-assignedTo : int
-requiredRole : string
+execute(actorId : int) : bool
+assignTo(participantId : int) : void
+getAssignee() : int
+getRequiredRole() : string
+release() : void
+reopen() : bool
+getChildren() : const vector<shared_ptr<WorkItem>>&
+isComplete() : bool
+isAvailable() : bool
CompositeWorkItem
-children : vector<shared_ptr<WorkItem>>
-sequential : bool
+execute(actorId : int) : bool
+add(item : shared_ptr<WorkItem>) : void
+getChildren() : const vector<shared_ptr<WorkItem>>&
+release() : void
+isComplete() : bool
+createIterator() : unique_ptr<WorkItemIterator>
+createAvailableIterator() : unique_ptr<WorkItemIterator>
+isAvailable() : bool
root 1
children 0..*
«create»
WorkItemIterator
+first() : void
+next() : void
+isDone() : bool
+current(item) : shared_ptr<WorkItem>
DepthFirstIterator
-items : vector<shared_ptr<WorkItem>>
-index : size_t
+DepthFirstIterator(root : const WorkItem&)
-collect(item : const WorkItem&) : void
+first() : void
+next() : void
+isDone() : bool
+current(item) : shared_ptr<WorkItem>
AvailableWorkIterator
-inner : DepthFirstIterator
+AvailableWorkIterator(root : const WorkItem&)
-skipUnavailable() : void
+first() : void
+next() : void
+isDone() : bool
+current(item) : shared_ptr<WorkItem>
inner
WorkItem
#id : int
#name : string
#status : string
+WorkItem(id : int, name : const string&, status : const string&)
+execute(actorId : int) : bool
+getId() : int
+getName() : string
+getStatus() : string
+isAvailable() : bool
+isComplete() : bool
+getChildren() : const vector<shared_ptr<WorkItem>>&
+release() : void
+reopen() : bool
+assignTo(participantId : int) : void
+getAssignee() : int
+getRequiredRole() : string
WorkflowDefinition
-name : string
-version : int
-description : string
-approverRole : string
-approvalTimeout : int
-reminderInterval : int
-stages : vector<pair<string, vector<string>>>
-sequentialStages : vector<bool>
-requestFields : vector<pair<string, string>>
-validationChain : shared_ptr<ValidationHandler>
-factory : shared_ptr<WorkflowFactory>
-manualReviewAttribute : string
-manualReviewFrom : double
+setDescription(text : const string&) : WorkflowDefinition&
+setApproverRole(role : const string&) : WorkflowDefinition&
+setApprovalTimeout(seconds : int) : WorkflowDefinition&
+setReminderInterval(seconds : int) : WorkflowDefinition&
+addRequestField(key : const string&, prompt : const string&) : WorkflowDefinition&
+addStage(name : const string&, tasks : const vector<string>&, sequential : bool) : WorkflowDefinition&
+setManualReviewRule(attribute : const string&, from : double) : WorkflowDefinition&
+setValidationChain(chain : shared_ptr<ValidationHandler>) : WorkflowDefinition&
+setFactory(factory : shared_ptr<WorkflowFactory>) : WorkflowDefinition&
+getName() : string
+getVersion() : int
+getStages() : const vector<pair<string, vector<string>>>&
+getValidationChain() : shared_ptr<ValidationHandler>
+getFactory() : shared_ptr<WorkflowFactory>
+getApproverRole() : string
+getApprovalTimeout() : int
+getRequestFields() : const vector<pair<string, string>>&
+isStageSequential(index : size_t) : bool
+getManualReviewAttribute() : string
+getManualReviewFrom() : double
definition 1
currentState 1
WorkItemState
+getStateName() : string
+pending(workflow : WorkflowInstance&) : void
+accepted(workflow : WorkflowInstance&) : void
+rejected(workflow : WorkflowInstance&) : void
+approved(workflow : WorkflowInstance&) : void
+denied(workflow : WorkflowInstance&, message : const string&) : void
+completed(workflow : WorkflowInstance&) : void
+suspended(workflow : WorkflowInstance&) : void
+illegalState(workflow : WorkflowInstance&, newState : const string&) : void
approvalPolicy 1
ApprovalPolicy
#approverRole : string
+ApprovalPolicy(approverRole : const string&)
+canApprove(wf : const WorkflowInstance&, actor : const Participant&) : bool
+shouldAutoApprove(wf : const WorkflowInstance&) : bool
+describe() : string
TimeoutApprovalPolicy
-timeoutSeconds : int
+TimeoutApprovalPolicy(approverRole : const string&, seconds : int)
+shouldAutoApprove(wf : const WorkflowInstance&) : bool
+describe() : string
StrictApprovalPolicy
+StrictApprovalPolicy(approverRole : const string&)
+canApprove(wf : const WorkflowInstance&, actor : const Participant&) : bool
+shouldAutoApprove(wf : const WorkflowInstance&) : bool
+describe() : string
PendingState
+getStateName() : string
+accepted(workflow : WorkflowInstance&) : void
+rejected(workflow : WorkflowInstance&) : void
AcceptedState
+getStateName() : string
+approved(workflow : WorkflowInstance&) : void
+denied(workflow : WorkflowInstance&, message : const string&) : void
ApprovedState
+getStateName() : string
+completed(workflow : WorkflowInstance&) : void
+suspended(workflow : WorkflowInstance&) : void
SuspendedState
+getStateName() : string
+accepted(workflow : WorkflowInstance&) : void
RejectedState
+getStateName() : string
DeniedState
+getStateName() : string
CompletedState
+getStateName() : string
AutoApproveCommand
+execute() : bool
+describe() : string
RejectCommand
-reason_ : string
+execute() : bool
+describe() : string
SuspendCommand
+execute() : bool
+undo() : bool
+describe() : string
ResumeCommand
+execute() : bool
+undo() : bool
+describe() : string
CompleteWorkItemCommand
-itemId_ : int
+execute() : bool
+undo() : bool
+describe() : string
EditReportCommand
-text_ : string
-before_ : unique_ptr<ReportMemento>
+execute() : bool
+undo() : bool
+describe() : string
invoker 1
history_ 0..*
WorkflowAction
#engine_ : WorkflowEngine&
#workflowId_ : int
#actorId_ : int
+execute() : bool
+undo() : bool
+describe() : string
#getWorkflowId() : int
#target() : shared_ptr<WorkflowInstance>
ApproveCommand
+execute() : bool
+describe() : string